M serving, multi-instance load balancing, vLLM, NLMS, cluster gateway, open-source systems

DIO: A Non-Invasive Control Plane for Multi-Instance LLM Serving over Stock vLLM

Keyush Nisar[1*], Krishil Parikh[1] and Krisha Maisheri[1]

[1*]Department of Computer Engineering, Dwarkadas J. Sanghvi College of Engineering, Mumbai, India.

*Corresponding author(s). E-mail(s): Nisarkeyush3@gmail.com;
Contributing authors: Krishilparikh75@gmail.com;
KrishaMaisheri16@gmail.com;

Abstract

Deployments often place several stock vLLM (or OpenAI-compatible) instances behind Round-Robin or connection-count balancers. That works poorly when backends differ in *effective* service cost—queue depth, interference, or a temporarily slow peer—even when GPU SKUs match. We present **DIO**/`dio-serve`, a non-invasive OpenAI-compatible gateway that ranks backends with dual-timescale NLMS slopes learned from **request end-to-end latency and token counts** (plus optional VRAM hints and in-gateway queue depth), not full GPU SM/power telemetry.

Honest scope. Absolute latency prediction is weak (MAPE ∼90–130% on dual-T4); DIO is useful only as a *relative ranker* under measurable service skew. On dual Tesla T4 + Qwen2.5-3B + stock vLLM ($\boldsymbol{n}$**=10** seeds): (i) when backends are identical, NLMS p99 is *not* better than Round-Robin (within ∼3%, slightly worse); (ii) when one peer's observed e2e is delay-inflated $\times$**2**, NLMS cuts p99 by **48.3%±0.7%** vs. RR and beats $\boldsymbol{d}$**=2** RLS. We release open code and multi-seed artifacts. Multi-SKU fleets are out of scope for this paper.

1 Introduction

Practitioners commonly run *multiple* stock vLLM [1] (or SGLang/TGI [2, 3]) instances and put Nginx, Kubernetes Services, or Ray Serve [11] in front with Round-Robin or least-connections. Single-node engines optimize continuous batching; they do not solve *which backend* should take the next request. Naive balancers treat backends as

equal. In practice effective cost still diverges—pending queue, batching noise, host interference, or a peer that is temporarily slow—even on identical GPU SKUs. The result is avoidable tail latency [5].

We design **DIO** (`dio-serve`) as a thin, non-invasive OpenAI-compatible gateway for this multi-instance pattern. DIO is *not* a full GPU telemetry stack (no continuous NVML SM/power/temperature loop in the default path). It learns from what a stock engine already returns: **end-to-end request latency and token usage**, plus **in-gateway pending counts**, optional static/API VRAM hints, and a cheap prefix-hash affinity bonus.

What NLMS can and cannot do (preview of results).

Dual-timescale NLMS [7] does **not** magically invent wins when backends are equal: on homogeneous dual-T4, NLMS p99 is within a few percent of Round-Robin and slightly worse (Section 4.3). Absolute $\hat{y}$ is poor (MAPE $\sim$90–130%). Where NLMS helps is when backends differ in *observed* service cost: under a controlled $\times 2$ e2e inflation on one real peer, NLMS reduces p99 by $48.3\% \pm 0.7\%$ vs. RR and beats d=2 RLS (Section 4.6). That is ranking under skew—not GPU-SKU wizardry and not accurate ms forecasting.

Contributions.

1. **Deployable multi-instance gateway.** Open-source `dio-serve` wraps stock OpenAI-compatible engines without patches; scheduling overhead $\sim$10–15 μs.
2. **Online relative ranking under weak absolute models.** Dual-timescale NLMS + joint cost; admission decoupled from absolute $\hat{y}$ when MAPE is high.
3. **Honest multi-seed evaluation on real dual-T4 + vLLM.** Homogeneous (no free lunch) and service-time skew (clear p99 gains), plus NLMS vs. RLS/RR/LL under a fixed cost function. Multi-SKU fleets are out of scope.

2 Background and related work

2.1 Single-node engines

vLLM [1] popularized PagedAttention and continuous batching. SGLang [2] improves structured decoding and prefix reuse. TGI [3] targets production APIs. Disaggregated prefill/decode systems [8–10] optimize pipeline stages. These systems excel on one node; multi-worker routing is external.

2.2 Cluster orchestrators

Ray Serve [11] and Triton [12] emphasize actors and model repositories with coarse metrics. Orca [13] advances iteration-level batching on homogeneous clusters. AlpaServe [14] and CoCoServe [15] focus on placement and multiplexing. Mélange [6] targets heterogeneous capacity tables. NexusSched [4] combines predictive routing with engine co-design and multi-feature predictors ($O(d^2)$ updates). Llumnix [16] migrates live KV state. LoongServe [17] targets long-context elastic parallelism. DIO

complements these as a *non-invasive request-level front end*: dual-timescale scalar NLMS, joint cost, and stock engines only.

2.3 Adaptive prediction

NLMS stability and step-size practice follow adaptive filter theory [7]. Queue wait uses a batch-amortized Little-style heuristic [18]. SJF-style ranking motivates predicted service routing [19]. Roofline [20] inspires soft memory ceilings recast as routing penalties.

3 System design

3.1 Architecture

DIO separates a **control plane** from a **data plane** of unmodified engines (Fig. 1). Clients send OpenAI-compatible `/v1/chat/completions` to DIO. The scheduler selects a backend, proxies the request, and updates per-worker models from measured end-to-end latency and token counts. Product path: `dio-serve` (FastAPI + Python scheduler). Research path: Go manager with gRPC workers (optional).

Telemetry used in practice.

The default closed loop uses: (i) wall-clock e2e latency through the gateway; (ii) token counts from `usage` or a tokenizer estimate; (iii) per-backend `pending` queue depth inside DIO. Soft/hard VRAM terms exist in the cost function but require free-VRAM to be set (config, API, or optional hooks); stock vLLM OpenAI responses do **not** automatically stream full GPU telemetry into DIO. We therefore do not claim continuous awareness of SM utilization, power, or temperature.

3.2 Dual-timescale NLMS latency model

For worker w and approximate token count N, we model

$$\hat{y} = s_w N + b_w, \tag{1}$$

where s_w is ms/token and b_w is fixed overhead. On completion with residual $e = y - \hat{y}$, NLMS updates

$$s \leftarrow s + \mu \frac{e}{N}, \qquad b \leftarrow b + \mu_b e, \tag{2}$$

with dual rates μ_{fast}=0.1, μ_{slow}=0.01, μ_b=0.005, and

$$s^{\text{eff}} = \alpha s^{\text{fast}} + (1-\alpha) s^{\text{slow}}, \quad \alpha=0.8. \tag{3}$$

Cold-start priors are s_0=2.0 ms/token, b_0=150 ms. We distinguish (i) $O(1)$ *arithmetic* per update from (ii) time-to-usable ranking: typically **15–20 completions per worker** before slopes leave priors. Figure 2 illustrates dual-timescale response to an injected slope jump in simulation.

Token feature N prefers Hugging Face tokenizer counts plus planned `max_tokens`; fallback is $\lfloor |\text{prompt}|/4 \rfloor$ + `max_tokens`.

Algorithm 1 DIO worker selection (simplified)

Require: request r, workers W, admission mode

$N \leftarrow$ token feature(r)
$w^\star \leftarrow \perp$, $S_{\min} \leftarrow \infty$
for all $w \in W$ healthy and tier-feasible **do**
    $\hat{t}_w \leftarrow \text{Predict}(w, N)$
    if VRAM hard-blocked(w, N) **then continue**
    end if
    $S_w \leftarrow \text{wait} + \hat{t}_w + \text{tier} + \text{vram} - \text{cache}$
    if $S_w < S_{\min}$ **then** $S_{\min} \leftarrow S_w$, $w^\star \leftarrow w$
    end if
end for
if $w^\star = \perp$ or admission rejects **then return** HTTP 503
end if
return $w^\star$

3.3 Joint cost routing

For each healthy worker w,

$$S_w = \text{wait}_w + \hat{t}_w + \text{tierCost}_{r,w} + \text{vramCost}_w - \text{cacheBonus}_w, \tag{4}$$

with $\hat{t}_w = s_w^{\text{eff}} N + b_w$, $\text{wait}_w = (Q_w/B)\bar{t}_w$ (B=8), soft VRAM penalty when free VRAM <4 GB, hard exclusion when free VRAM <2.4 GB and N>1000, tier mismatch soft cost 500 ms (hard block for large-on-small), and optional 200 ms prefix-hash affinity bonus. Route to $\arg\min_w S_w$ in $O(|W|)$ time.

Admission vs. ranking.

Because absolute MAPE is high (Section 4.4), admission is *not* primarily "reject if $\min S_w >$ SLO" on raw $\hat{y}$. Modes: (i) `rank_only` (VRAM/tier hard blocks only; default for routing A/B); (ii) `empirical` (rolling observed latency percentiles); (iii) `absolute` (legacy, diagnostic). Metrics track disagreement between absolute and active modes.

3.4 Implementation

`dio serve -b e0=URL0 -b e1=URL1` fronts stock engines. Strategies: `nlms`, `rls`, `round_robin`, `least_loaded`, `static`. Debug metrics expose slopes, MAPE, routes, and admission counters. Artifacts and scripts: workshop validation suite under the public repository.

4 Experimental evaluation

4.1 Methodology

Regimes (non-interchangeable).

1. **Regime A — homogeneous dual-T4 (real):** 2× Tesla T4, stock vLLM, Qwen2.5-3B-Instruct, HF tokenizer, `max_tokens`=32, n=10 seeds × 30 requests.
2. **Regime B — legacy Locust pilot (secondary):** longer-decode ShareGPT/arXiv/Azure, single seed, different setup; absolute seconds not mixed with Regime A ms.
3. **Regime C — service-time skew:** (i) CPU multi-seed simulation n=10; (ii) real dual-T4 with e1 behind delay proxy ×2, n=10 (slow peer, same SKU).

Baselines.

Under the *same* joint cost and the same engines: Round-Robin (RR), Least-Loaded (LL), and RLS (d=2, λ=0.99). Primary metrics: p50/p99 e2e mean±std with approximate 95% CI half-widths; MAPE; fraction to backend e0 (unthrottled/fast). We do not claim a matched multi-node Orca or vLLM-distributed bake-off.

Simulation vs. real.

"Sim" denotes library slope-skew or calibrated mocks. "Real" denotes dual-T4 + stock vLLM. Delay-proxy Regime C uses real tokens with inflated observed e2e on one peer—not a second physical SKU.

4.2 Control-plane overhead

On the T7 scalability microbench with many concurrent mock workers, instrumented scheduling completes in approximately **14 μs** per request (worker scan, cost evaluation, prefix hash). Per-worker state is ∼256 B; the control plane is not the bottleneck relative to LLM decode. Figure 3 shows the control-plane stress plot (e2e p99 is dominated by synthetic worker delay).

4.3 Regime A: homogeneous dual-T4 (NLMS is not better than RR)

Table 1 is the primary homogeneous multi-seed matrix. On *identical* backends, **NLMS does not beat Round-Robin on mean p99** (NLMS is slightly worse: $-2.6\%\pm1.0\%$ "improvement"). This is the correct outcome if ranking is driven by learned service cost and workers truly look alike. Useful secondary signals: NLMS keeps tight p99 std (31 ms) while Least-Loaded is sticky (100% to e0, p99 std 430 ms); RLS mis-routes (frac e0 0.12).

4.4 MAPE vs. relative ranking

NLMS MAPE on dual-T4 is **high** ($123 \pm 6\%$ with tokenizer). HF tokenizer vs. byte heuristic (W2, n=3): MAPE $117.5\pm10.9\%$ vs. $89.7\pm4.9\%$—tokenizer is **worse**, so error

Table 1 Regime A (real, primary). Dual-T4 + vLLM + Qwen2.5-3B, HF tokenizer, n=10 × 30, `max_tokens`=32. Mean±std.

Strategy	p99 (ms)	MAPE (%)	frac e0	vs RR
NLMS	3413 ± 31	123 ± 6	0.45 ± 0.04	$-2.6 \pm 1.0\%$
RLS (d=2)	3466 ± 11	132 ± 6	0.12 ± 0.02	$-4.2 \pm 0.6\%$
Round-Robin	3326 ± 17	88 ± 2	0.50 ± 0.00	—
Least-Loaded	3313 ± 430	141 ± 13	1.00 ± 0.00	$+0.4 \pm 13\%$

Table 2 Regime B (legacy single-seed Locust pilot). Not multi-seed; different setup from Table 1.

Trace	Strat.	p99 (s)	RPS	Fail %
ShareGPT	NLMS	**67**	**0.37**	0
ShareGPT	RR	81	0.29	0
arXiv	NLMS	52	**0.36**	0
arXiv	RR	51	0.30	0
Azure	NLMS	48	0.33	0
Azure	RR	47	0.36	0

Table 3 Regime C (simulation). Slope-skew dual workers, n=10 × 200.

Metric	NLMS	RR	Notes
Frac. to fast	0.975 ± 0.000	0.500 ± 0.000	
p99 (sim units)	1411 ± 34	2287 ± 49	
p99 vs RR	$38.3\% \pm 2.0\%$		

is structural (linear model, cold start, batching), not mere token counting. Routing uses $\arg\min S_w$; ranking under skew is the success criterion (Regime C).

4.5 Regime B: legacy Locust pilot (secondary)

Table 2 retains a single-seed Locust pilot (longer decode, four logical workers). Absolute seconds are **not** comparable to Regime A milliseconds.

4.6 Regime C: slope skew

Simulation *(n=10).*

With distinct true (b, s) pairs, NLMS places 97.5% of traffic on the fast worker and cuts p99 by $38.3\% \pm 2.0\%$ vs. RR (Table 3). RLS fails to rebalance (frac fast 0.40 ± 0.18). Local `pick()` overhead: NLMS $\approx 9.6\,\mu$s, RLS $\approx 9.4\,\mu$s at d=2.

Table 4 Regime C (real GPU, primary). Dual-T4 + delay-proxy ×2, n=10 × 30, HF tokenizer.

Strategy	p99 (ms)	frac e0	vs RR	MAPE (%)
NLMS	$\mathbf{3427 \pm 38}$	0.47 ± 0.08	$\mathbf{48.3 \pm 0.7}\%$	126 ± 12
RLS (d=2)	5087 ± 1505	0.37 ± 0.19	$23.2 \pm 22.9\%$	119 ± 10
Round-Robin	6632 ± 37	0.50 ± 0.00	—	102 ± 3

Real dual-T4 service-time skew (n=10).

We inflate e1 wall-clock e2e by ×2 via `latency_delay_proxy` while still serving real tokens on a real T4—a controlled *slow peer*, not a second GPU SKU. Table 4: NLMS p99 3427 ± 38 ms vs. RR 6632 ± 37 ms ($48.3\% \pm 0.7\%$ improvement). RLS is weaker and noisier ($23.2\% \pm 22.9\%$). Mean frac to e0 under NLMS is only modest (0.47 ± 0.08): NLMS is **not** a dramatic sticky "always hit the fast GPU" policy in this sequential short-probe setting; the supported claim is **better p99 under skew than RR/RLS**, not sim-style 97.5% splits.

Longer decode.

Homogeneous dual-T4, `max_tokens`=128, n=3 × 20: NLMS 13835 ± 73 ms vs. RR 13821 ± 88 ms (tied), matching Regime A qualitative message.

4.7 Ablations and resilience

On an L4 stressed long-prompt microbench, removing VRAM admission elevates hard failures (∼23%); removing queue wait raises tail latency; removing tiers hurts mixed-capability mixes (Fig. 4). Cold-start and overload admission behaviors are summarized in Fig. 5: under intentional overload, DIO returns HTTP 503 rather than unbounded queueing.

5 Discussion and limitations

Is NLMS "good at routing"?

Honestly: **only when backends differ in observed service cost.** On matched dual-T4 workers, NLMS is no better than RR on mean p99 (Section 4.3). Under a slow peer, NLMS improves p99 substantially vs. RR and RLS (Section 4.6), but mean traffic share to the faster backend stays modest. Absolute $\hat{y}$ is not trustworthy (high MAPE). DIO should be sold as a *lightweight multi-instance ranker under skew*, not as a GPU-omniscient scheduler.

Limitations.

1. **Telemetry depth.** Default loop is e2e latency + tokens + gateway pending; not continuous GPU SM/power telemetry.
2. **Skew model.** Real skew uses delay-proxy on identical T4s, not physical multi-SKU fleets.

3. **Baselines.** RR/LL/RLS on the same vLLM pair; not multi-node Orca/vLLM-distributed.
4. **Workload.** Short sequential chat probes dominate multi-seed cells; concurrent ShareGPT multi-seed is incomplete.
5. **Out of scope.** Prefill/decode disaggregation co-design, multimodal split inference.

6 Conclusion

DIO (`dio-serve`) is a non-invasive multi-instance gateway for stock vLLM-class engines: dual-timescale NLMS ranking from request e2e latency and tokens, joint cost routing, and admission that does not pretend absolute $\hat{y}$ is accurate. Multi-seed dual-T4 results show **no free lunch when backends match** and **clear p99 gains when one peer is effectively slower**. That is a deployable systems contribution for common multi-vLLM deployments—without requiring multi-SKU hardware or full GPU telemetry.

Acknowledgements. The authors thank colleagues who provided feedback on earlier drafts.

Declarations

- **Funding.** No external funding was received for this work.
- **Conflict of interest/Competing interests.** The authors declare that they have no competing interests.
- **Ethics approval and consent to participate.** Not applicable (no human subjects or personal data).
- **Consent for publication.** Not applicable.
- **Data availability.** Aggregated experimental results are included in the repository under `results_workshop_final/` and related result directories. Raw engine logs can be regenerated with the public scripts.
- **Materials availability.** Not applicable.
- **Code availability.** Source code for `dio-serve`, validation suites (`run_workshop_final_suite.py`, delay-proxy harness), and paper artifacts are available in the project repository (https://github.com/nisaral/DIO).
- **Author contribution.** K.N. led system design, implementation, experiments, and manuscript drafting. K.P. and K.M. contributed to evaluation, analysis, and manuscript revision. All authors approved the final manuscript.

Appendix A Notation summary

- $s^{\text{fast}}, s^{\text{slow}}, s^{\text{eff}}$: dual-timescale and blended slopes (ms/token).
- b_w: intercept (ms).
- N: token feature; y, $\hat{y}$: actual and predicted e2e latency (ms).
- S_w: joint routing cost; Q_w: pending queue depth; B: batch amortization constant.

References

[1] Kwon, W., *et al.*: Efficient memory management for large language model serving with PagedAttention. In: SOSP (2023)

[2] Zheng, L., et al.: SGLang: Efficient Execution of Structured Language Model Programs (2023)

[3] Hugging Face: Text Generation Inference. https://github.com/huggingface/text-generation-inference (2023)

[4] Zhang, Y., Chen, Y., Mo, X., Xi, A., Li, J., Wu, W.: A Predictive and Synergistic Two-Layer Scheduling Framework for LLM Serving (NexusSched) (2025)

[5] Dean, J., Barroso, L.A.: The tail at scale. Communications of the ACM **56**(2) (2013)

[6] et al.: Mélange: Cost-Efficient LLM Serving on Heterogeneous GPUs. arXiv preprint (2024)

[7] Haykin, S.: Adaptive Filter Theory, 4th edn. Prentice Hall, ??? (2002)

[8] Zhong, Y., *et al.*: DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In: OSDI (2024)

[9] Patel, P., et al.: Splitwise: Efficient Generative LLM Inference Using Phase Splitting. arXiv preprint (2024)

[10] Agrawal, A., *et al.*: Sarathi-Serve: Efficiently serving large language models via hybrid scheduling. In: OSDI (2024)

[11] Ray Project: Ray Serve. https://docs.ray.io/en/latest/serve/ (2024)

[12] NVIDIA Corporation: Triton Inference Server (2024)

[13] Yu, G.-I., *et al.*: Orca: A distributed serving system for Transformer-based generative models. In: OSDI (2022)

[14] Li, Z., *et al.*: AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In: OSDI (2023)

[15] Wu, J., et al.: Unlock the Potential of Fine-grained LLM Serving via Module-wise Autoscaling (CoCoServe) (2025)

[16] Wu, B., et al.: Llumnix: Dynamic Scheduling for Large Language Model Serving (2024)

[17] Wu, B., Liu, S., Zhong, Y., Sun, P., Liu, X., Jin, X.: LoongServe: Efficiently

serving long-context large language models with elastic sequence parallelism. In: SOSP (2024)

[18] Little, J.D.C.: A proof for the queuing formula: $L = \lambda W$. Operations Research (1961)

[19] Coffman, E.G., Denning, P.J.: Operating Systems Theory. Prentice-Hall, ??? (1973)

[20] Williams, S., Waterman, A., Patterson, D.: Roofline: An insightful visual performance model for multicore architectures. Communications of the ACM **52**(4) (2009)

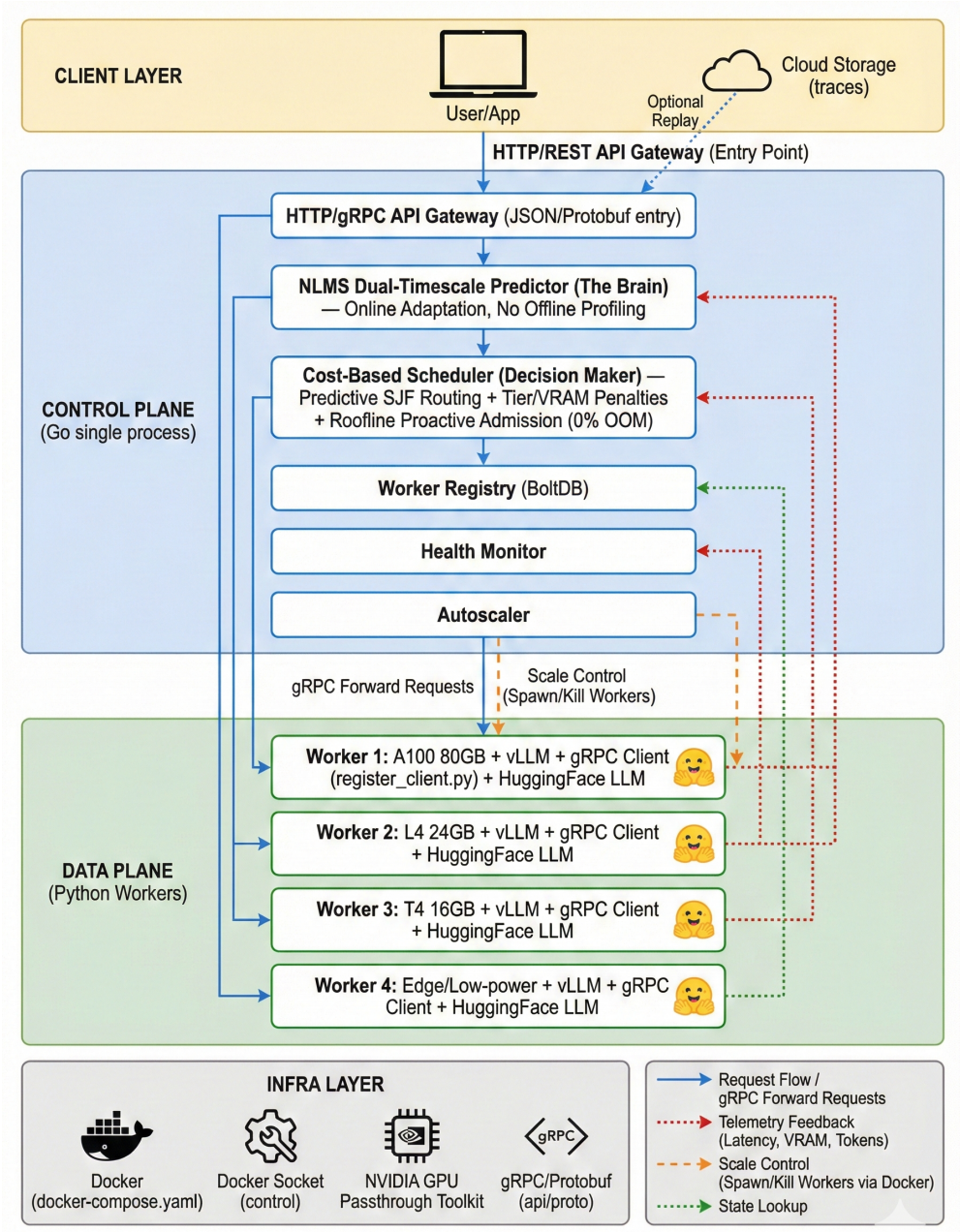

Fig. 1 DIO as a multi-instance gateway: OpenAI-compatible front door, dual-timescale NLMS from e2e latency/tokens, joint cost router, stock vLLM backends.

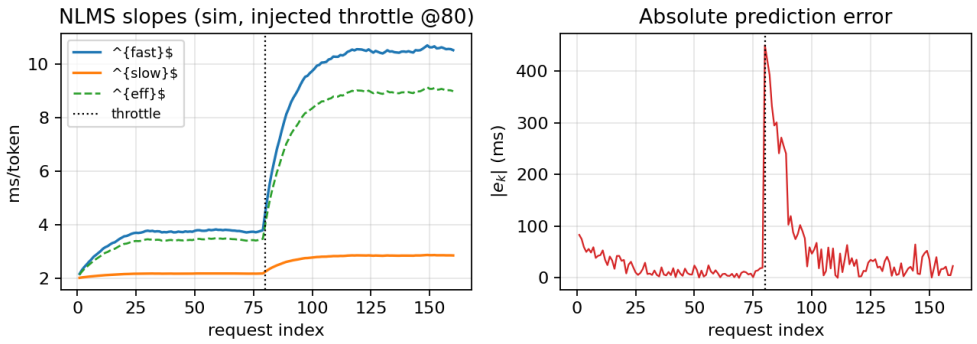

Fig. 2 NLMS dual-timescale adaptation under an injected true-slope jump at request 80 (simulation). s^{fast} reacts first; s^{slow} lags. Labeled synthetic illustration, not dual-T4 wall-clock traces.

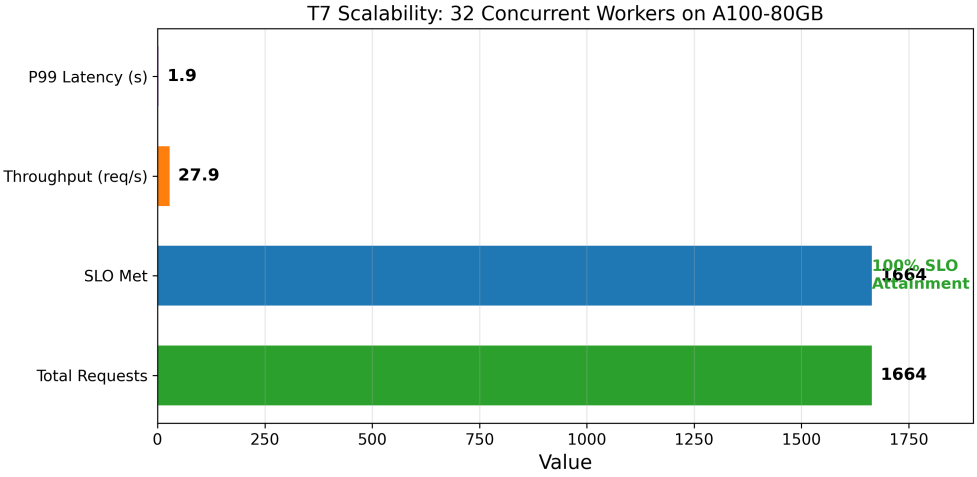

Fig. 3 Control-plane scalability microbench (T7). Scheduling overhead remains $\sim$14 μs; e2e latency in the plot is synthetic worker delay.

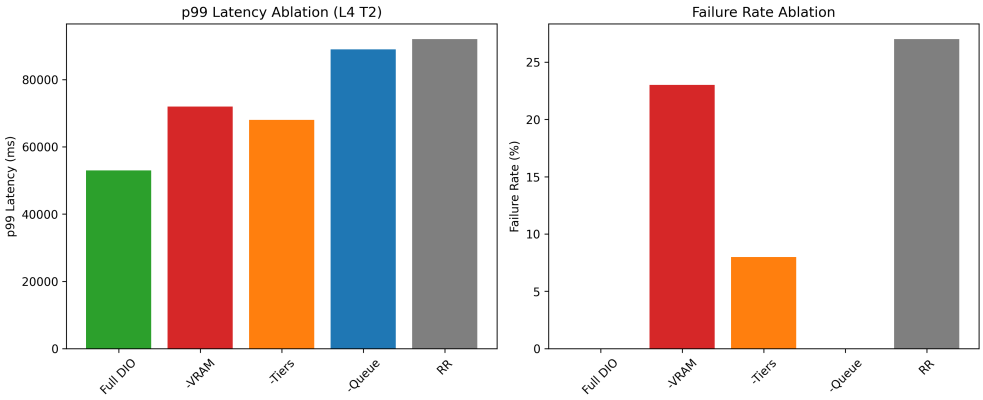

Fig. 4 Cost-term ablation (L4 microbench). Full DIO keeps failures low and tail controlled.

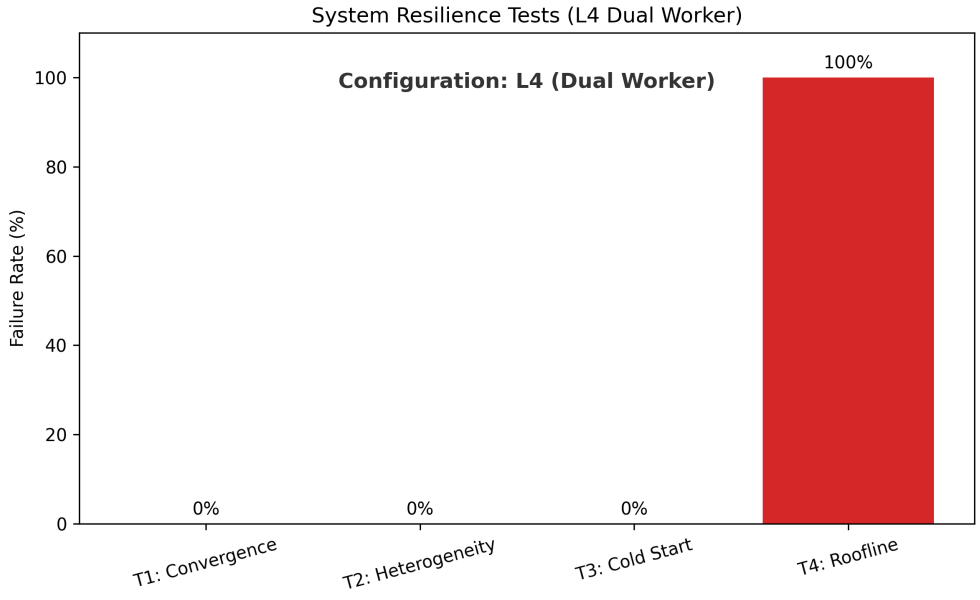

Fig. 5 Resilience microbenchmarks (L4 suite). Overload test exercises admission (503), not throughput maximization.